

Artificial Intelligence for Digital Characters

Solution Exercise 2

Hand-out Date: 8 April 2026



Goals

- Understanding the fundamentals and methodologies of text embedding and response generation.
- Understanding approaches for adapting Large Language Models (LLMs), such as model fine-tuning and prompt engineering.
- Understanding multimodal data processing and personality infusion basics and techniques.
- Integrating an LLM into a Unity project to enable the avatar to generate a response given the transcription of the user input.

Resources

The lecture slides, exercise slides, and Unity template are accessible via Moodle.

Tasks

1. Multiple Choice Questions

a) False. The nucleus sampling (Top- p) strategy randomly picks from the smallest subset with a cumulative mass greater than p . By contrast, the top- k sampling strategy will randomly pick from the k most likely words as responses.

b) False. While providing clear instructions is beneficial, overly detailed and restrictive prompts can sometimes reduce the model's creativity and flexibility, leading to less useful or overly rigid responses. Effective prompt engineering balances specificity with adaptability.

c) False. PEFT does not necessarily reduce the total number of model parameters but instead fine-tunes only a small subset of them (e.g., LoRA) while keeping the majority of the LLM's parameters frozen. This approach enables efficient adaptation with lower computational cost.

d) False. In-context learning does not involve training the model with prompt-response pairs; instead, it refers to the model's ability to learn and adapt to patterns within a given prompt during inference without updating its weights. The model utilizes the provided context to generate relevant responses based on its pre-trained knowledge.

e) True. RAG improves a language model's performance by supplementing its responses with up-to-date or domain-specific information retrieved from external sources. This approach helps overcome the limitations of static pre-trained knowledge, allowing the model to generate more informed and contextually relevant answers.

f) False. While both Mean Opinion Score (MOS) and A/B testing are qualitative evaluation methods, they differ in their approach. MOS involves users rating a single chatbot's responses on a numerical scale, assessing overall quality. In contrast, A/B testing relies on pairwise comparisons, where users choose between two chatbot versions to determine which performs better.

g) True. A chatbot with multi-modal inputs and outputs can improve user experience by leveraging user state modeling. By analyzing inputs like voice tone, facial expressions, or text sentiment, the chatbot can adjust its responses to be more empathetic, context-aware, and personalized, leading to more natural and engaging interactions.

h) False. While late fusion preserves individual modality representations by com-

binning them at a later stage, it is not always superior to early fusion. Early fusion, which integrates modalities at an earlier stage, can capture cross-modal interactions more effectively, potentially leading to better performance in certain tasks. The choice between early and late fusion depends on the specific application and the nature of the data.

i) False. While the five-factor model (openness, conscientiousness, extraversion, agreeableness, and neuroticism) is widely used for modeling human personality, chatbot personality is better represented by the five traits: artificiality, conscientiousness, vibrancy, civility, and neuroticism.

2. Text Representations and Fine-Tuning Techniques

a) Bag of Words (BoW):

- **Concept:** Represents text by counting word occurrences, ignoring word order and context.
- **Advantage:** Simple and efficient for tasks like document classification.
- **Limitation:** Fails to capture semantic meaning or relationships between words (e.g., "cat" and "kitten" are treated as separate).

Word Embeddings (e.g., Word2Vec, GloVe):

- **Concept:** Represents words as dense vectors that capture semantic meaning based on context.
- **Advantage:** Captures semantic relationships between words (e.g., "king" and "queen" are closer in vector space).
- **Limitation:** Requires large corpora for training and may struggle with polysemy (words with multiple meanings).

b) Parameter-Efficient Fine-Tuning (PEFT):

- **Concept:** PEFT involves adapting pre-trained models to specific tasks with minimal changes to the model's parameters. Instead of fine-tuning all the parameters, PEFT techniques focus on updating only a small subset of parameters, such as adding task-specific layers or fine-tuning only certain layers, thus reducing the computational and storage cost.
- **When to Apply:** PEFT is beneficial when using large pre-trained models (e.g., BERT, GPT) for downstream tasks, especially when resources for training are limited, or when trying to maintain the general knowledge of a pre-trained model while focusing on a specific task.

Pruning:

- **Concept:** Pruning reduces the size of a neural network by removing less important weights, neurons, or connections. This optimization technique helps compress the model while maintaining its performance by eliminating redundant or unimportant parts of the network.
- **When to Apply:** Pruning is useful when the goal is to reduce a model's memory footprint and computational cost, especially for deployment in resource-constrained environments, such as edge devices or mobile platforms.

c) Step-1: Apply Top- k Strategy. The top- k sampling strategy selects the k tokens with the highest probabilities. In the case with $k = 3$, we simply identify the 3 words with the largest probability values as the final sampling pool:

Rank 1: strawberries ($P = 0.35$)

Rank 2: fruit ($P = 0.25$)

Rank 3: yogurt ($P = 0.15$)

Step-2: Apply Top- p (Nucleus) Strategy. The top- p strategy selects the smallest possible set of top-ranked tokens whose cumulative probability exceeds or equals the threshold p . In this case, we use $p = 0.8$. First, rank the words by probability in descending order (already done) and compute the cumulative probabilities step-by-step:

- Add “strawberries”:

Cumulative $P = 0.35$

Since $0.35 < 0.80$, continue.

- Add “fruit”:

Cumulative $P = 0.35 + 0.25 = 0.60$

Since $0.60 < 0.80$, continue.

- Add “yogurt”:

Cumulative $P = 0.60 + 0.15 = 0.75$

Since $0.75 < 0.80$, continue.

- Add “cereal”:

Cumulative $P = 0.75 + 0.10 = 0.85$

Since $0.85 \geq 0.80$, the threshold is crossed. We stop here.

Using the **top-0.8 strategy**, the final sampling pool includes: strawberries, fruit, yogurt, and cereal.

d) 1. Chain-of-Thought Prompting: Chain-of-Thought (CoT) prompting encourages the model to explain intermediate steps before reaching a final answer.

Improved Prompt: *”To find the square root of 144, let’s follow these steps: 1. Recall that the square root of a number x is a value y such that $y^2 = x$. 2. Identify a number y such that $y \times y = 144$. 3. What is this number?”*

This prompt forces the model to engage in structured reasoning, reducing errors and increasing interpretability.

2. Self-Ask Prompting: Self-Ask Prompting decomposes a problem into a sequence of sub-questions to guide the model through logical steps.

Improved Prompt: *"Let's break this problem into smaller steps: 1. What does it mean to take the square root of a number? 2. What number squared equals 144? 3. What is the final answer?"*

By explicitly structuring the solution process, Self-Ask Prompting ensures that the model answers each sub-question before reaching a conclusion, improving accuracy and clarity.

e) Step-1: Apply the Logarithm to the Probability. We want to compute the gradient of one summand: $\frac{\partial}{\partial v_c} \log P(w_o|w_c; \theta)$. First, substitute the probability formula into the logarithm:

$$\frac{\partial}{\partial v_c} \log P(w_o|w_c; \theta) = \frac{\partial}{\partial v_c} \log \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Step-2: Split the Logarithm and Differentiate the First Term. Using the properties of logarithms, separate the numerator and denominator:

$$= \frac{\partial}{\partial v_c} \log \exp(u_o^T v_c) - \frac{\partial}{\partial v_c} \log \sum_{w \in V} \exp(u_w^T v_c)$$

For the first term, the log and exp cancel out, leaving the derivative of $u_o^T v_c$ with respect to v_c , which evaluates to the observed vector u_o .

Step-3: Apply the Chain Rule to the Second Term. To differentiate the second term, use the chain rule $\frac{\partial}{\partial x} f(g(x)) = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$. Applying this yields:

$$\frac{1}{\sum_{w \in V} \exp(u_w^T v_c)} \frac{\partial}{\partial v_c} \sum_{w \in V} \exp(u_w^T v_c)$$

Next, compute the derivative of the sum by pushing the derivative inside:

$$\frac{\partial}{\partial v_c} \sum_{w \in V} \exp(u_w^T v_c) = \sum_{w \in V} \frac{\partial}{\partial v_c} \exp(u_w^T v_c) = \sum_{w \in V} \exp(u_w^T v_c) u_w$$

Step-4: Combine Terms and Simplify. Substitute the result back into the full expression:

$$u_o - \frac{1}{\sum_{w \in V} \exp(u_w^T v_c)} \sum_{x \in V} \exp(u_x^T v_c) u_x$$

Rearrange the terms by bringing the scalar denominator inside the sum:

$$= u_o - \sum_{x \in V} \frac{\exp(u_x^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)} u_x$$

Recognize that the fractional term is equivalent to the probability $P(x|c)$:

$$= u_o - \sum_{x \in V} P(x|c) u_x$$

Conclusion: The final gradient is the observed vector u_o minus the expectation $\sum_{x \in V} P(x|c) u_x$. This form is easy to optimize, meaning you can do classic stochastic gradient descent (SGD) to minimize the objective $J(\theta)$ and thus maximize the data likelihood $L(\theta)$.

3. LLM Evaluation and Information Retrieval

a) Large Language Models (LLMs) can be improved through two key techniques: in-context learning and instruction tuning. These methods enhance model adaptability and response quality.

In-Context Learning

- **Definition:** The model learns from examples provided in the input prompt without updating its parameters.
- **Advantage:** Quick adaptation to new tasks without retraining.
- **Better suited scenarios:** Few-shot or zero-shot learning scenarios where new tasks need to be handled dynamically.

Instruction Tuning

- **Definition:** The model is fine-tuned using instruction-response pairs to better follow human instructions.
- **Advantage:** Improves generalization and adherence to user instructions.
- **Better suited scenarios:** Tasks requiring consistent, instruction-following behavior across various inputs.

b) Consider the following query and 3 candidate documents:

- Query: *deep learning transformers*
- Document 1: *Transformers have revolutionized deep learning by enabling long-range dependencies.*
- Document 2: *Deep neural networks are widely used in Chatbots.*
- Document 3: *Deep learning models have achieved state-of-the-art results in natural language processing.*

Recall tf-idf Formulation

The term frequency (tf) is defined as:

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d) & \text{if } \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The inverse document frequency (idf) is defined as:

$$\text{idf}_t = \log_{10} \frac{N}{\text{df}_t} \quad (2)$$

where N is the number of documents and df_t is the number of documents containing term t .

The tf-idf score is:

$$\text{tf-idf}(t, d) = \text{tf}_{t,d} \cdot \text{idf}_t \quad (3)$$

Step-1: Identify Terms from Query in Each Document

The query contains three terms: “deep”, “learning”, and “transformers”.

- “deep” appears in Documents 1, 2, and 3
- “learning” appears in Documents 1 and 3
- “transformers” appears in Document 1 only

Step-2: Calculate Term Frequency (tf)

For each term in each document:

$$\begin{aligned} \text{tf}(\text{“deep”}, \text{Doc}_1) &= 1 + \log_{10}(1) = 1 + 0 = 1 \\ \text{tf}(\text{“deep”}, \text{Doc}_2) &= 1 + \log_{10}(1) = 1 + 0 = 1 \\ \text{tf}(\text{“deep”}, \text{Doc}_3) &= 1 + \log_{10}(1) = 1 + 0 = 1 \\ \text{tf}(\text{“learning”}, \text{Doc}_1) &= 1 + \log_{10}(1) = 1 + 0 = 1 \\ \text{tf}(\text{“learning”}, \text{Doc}_2) &= 0 \text{ (not present)} \\ \text{tf}(\text{“learning”}, \text{Doc}_3) &= 1 + \log_{10}(1) = 1 + 0 = 1 \\ \text{tf}(\text{“transformers”}, \text{Doc}_1) &= 1 + \log_{10}(1) = 1 + 0 = 1 \\ \text{tf}(\text{“transformers”}, \text{Doc}_2) &= 0 \text{ (not present)} \\ \text{tf}(\text{“transformers”}, \text{Doc}_3) &= 0 \text{ (not present)} \end{aligned}$$

For the query:

$$\begin{aligned} \text{tf}(\text{“deep”}, \text{query}) &= 1 + \log_{10}(1) = 1 \\ \text{tf}(\text{“learning”}, \text{query}) &= 1 + \log_{10}(1) = 1 \\ \text{tf}(\text{“transformers”}, \text{query}) &= 1 + \log_{10}(1) = 1 \end{aligned}$$

Step-3: Calculate Inverse Document Frequency (idf)

$$\begin{aligned} \text{idf}(\text{“deep”}) &= \log_{10}(3/3) = \log_{10}(1) = 0 \\ \text{idf}(\text{“learning”}) &= \log_{10}(3/2) = \log_{10}(1.5) \approx 0.176 \\ \text{idf}(\text{“transformers”}) &= \log_{10}(3/1) = \log_{10}(3) \approx 0.477 \end{aligned}$$

Step-4: Calculate tf-idf for each term in each document

For Document 1:

$$\begin{aligned} \text{tf-idf}(\text{“deep”}, \text{Doc}_1) &= 1 \times 0 = 0 \\ \text{tf-idf}(\text{“learning”}, \text{Doc}_1) &= 1 \times 0.176 = 0.176 \\ \text{tf-idf}(\text{“transformers”}, \text{Doc}_1) &= 1 \times 0.477 = 0.477 \end{aligned}$$

For Document 2:

$$\begin{aligned}\text{tf-idf}(\text{"deep"}, \text{Doc}_2) &= 1 \times 0 = 0 \\ \text{tf-idf}(\text{"learning"}, \text{Doc}_2) &= 0 \times 0.176 = 0 \\ \text{tf-idf}(\text{"transformers"}, \text{Doc}_2) &= 0 \times 0.477 = 0\end{aligned}$$

For Document 3:

$$\begin{aligned}\text{tf-idf}(\text{"deep"}, \text{Doc}_3) &= 1 \times 0 = 0 \\ \text{tf-idf}(\text{"learning"}, \text{Doc}_3) &= 1 \times 0.176 = 0.176 \\ \text{tf-idf}(\text{"transformers"}, \text{Doc}_3) &= 0 \times 0.477 = 0\end{aligned}$$

For the query:

$$\begin{aligned}\text{tf-idf}(\text{"deep"}, \text{query}) &= 1 \times 0 = 0 \\ \text{tf-idf}(\text{"learning"}, \text{query}) &= 1 \times 0.176 = 0.176 \\ \text{tf-idf}(\text{"transformers"}, \text{query}) &= 1 \times 0.477 = 0.477\end{aligned}$$

Step-5: Calculate Cosine Similarity

The cosine similarity formula is:

$$\text{score}(q, d) = \sum_{t \in q} \frac{\text{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}}. \quad (4)$$

First, we calculate the normalization factor for the query:

$$\text{Query normalization} = \sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)} = \sqrt{0^2 + 0.176^2 + 0.477^2} = \sqrt{0.259} \approx 0.509$$

For the normalization factors of candidate documents, we need to compute the tf-idf for all terms occurring in the documents according to Eq. (4). For Document 1, the tf-idf results (the computation is similar to previous steps) are summarized in Table 1. Thus, we can compute the normalization factor for Document 1:

$$\begin{aligned}\text{Doc}_1 \text{ normalization} &= \sqrt{\sum_{d_i \in d_1} \text{tf-idf}^2(d_i, d_1)} \\ &= \sqrt{0.477^2 + 0.176^2 + 0.477^2 + 0^2 + 0.176^2 + 0.477^2 + 0.477^2 + 0.477^2 + 0.477^2} \\ &\approx 1.194\end{aligned}$$

Term	tf (Doc 1)	idf	tf-idf (Doc 1)
transformers	1.0	0.477	0.477
have	1.0	0.176	0.176
revolutionized	1.0	0.477	0.477
deep	1.0	0.000	0.000
learning	1.0	0.176	0.176
by	1.0	0.477	0.477
enabling	1.0	0.477	0.477
long-range	1.0	0.477	0.477
dependencies	1.0	0.477	0.477

Table 1: tf, idf, and tf-idf values for Document 1

For Document 2, the tf-idf results are summarized in Table 2. Thus, we can compute the normalization factor for Document 2:

$$\begin{aligned}
 \text{Doc}_2 \text{ normalization} &= \sqrt{\sum_{d_i \in d_2} \text{tf-idf}^2(d_i, d_2)} \\
 &= \sqrt{0^2 + 0.477^2 + 0.477^2 + 0.477^2 + 0.477^2 + 0.477^2 + 0.176^2 + 0.477^2} \\
 &\approx 1.182
 \end{aligned}$$

Term	tf (Doc 2)	idf	tf-idf (Doc 2)
deep	1.0	0.000	0.000
neural	1.0	0.477	0.477
networks	1.0	0.477	0.477
are	1.0	0.477	0.477
widely	1.0	0.477	0.477
used	1.0	0.477	0.477
in	1.0	0.176	0.176
chatbots	1.0	0.477	0.477

Table 2: tf, idf, and tf-idf values for Document 2

For Document 3, the tf-idf results are summarized in Table 3. Thus, we can compute the normalization factor for Document 3:

$$\begin{aligned}
 \text{Doc}_3 \text{ normalization} &= \sqrt{\sum_{d_i \in d_3} \text{tf-idf}^2(d_i, d_3)} \\
 &= \sqrt{0^2 + 0.176^2 + 0.477^2 + 0.176^2 + 0.477^2 + 0.477^2 + 0.477^2 + 0.176^2 + 0.477^2 + 0.477^2 + 0.477^2} \\
 &\approx 1.298
 \end{aligned}$$

Term	tf (Doc 3)	idf	tf-idf (Doc 3)
deep	1.0	0.000	0.000
learning	1.0	0.176	0.176
models	1.0	0.477	0.477
have	1.0	0.176	0.176
achieved	1.0	0.477	0.477
state-of-the-art	1.0	0.477	0.477
results	1.0	0.477	0.477
in	1.0	0.176	0.176
natural	1.0	0.477	0.477
language	1.0	0.477	0.477
processing	1.0	0.477	0.477

Table 3: tf, idf, and tf-idf values for Document 3

Now, we calculate the cosine similarity scores:

For Document 1:

$$\text{score}(q, \text{Doc}_1) = \frac{0 \times 0}{0.509 \times 1.194} + \frac{0.176 \times 0.176}{0.509 \times 1.194} + \frac{0.477 \times 0.477}{0.509 \times 1.194} \approx 0.4253$$

For Document 2:

$$\text{score}(q, \text{Doc}_2) = \frac{0 \times 0}{0.509 \times 1.182} + \frac{0.176 \times 0}{0.509 \times 1.182} + \frac{0.477 \times 0}{0.509 \times 1.182} = 0$$

For Document 3:

$$\text{score}(q, \text{Doc}_3) = \frac{0 \times 0}{0.509 \times 1.298} + \frac{0.176 \times 0.176}{0.509 \times 1.298} + \frac{0.477 \times 0}{0.509 \times 1.298} \approx 0.0469$$

Step-6: Rank Documents by Similarity Score

Rank	Document	Similarity Score
1	Document 1	0.4253
2	Document 3	0.0469
3	Document 2	0

Based on the cosine similarity scores, Document 1 has the highest similarity to the query, followed by Document 3, and then Document 2. This makes intuitive sense since Document 1 contains all query terms including the distinctive term “transformers,” Document 3 contains “deep” and “learning” but not “transformers,” and Document 2 only contains “deep” which has an idf of 0 (appearing in all documents, thus not discriminative).

c) The perplexity formula is defined as:

$$P = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_1, w_2, \dots, w_{i-1})}, \quad (5)$$

where P is the perplexity, N is the number of words in the response, and $P(w_i|w_1, w_2, \dots, w_{i-1})$ is the conditional probability of word w_i given all previous words.

Calculation for Response 1: “I like chatbot”

For this 3-word response, we need the following probabilities:

$$P(I|\langle \text{start} \rangle) = 0.5$$

$$P(\text{like}|I) = 0.6$$

$$P(\text{chatbot}|I \text{ like}) = 0.4$$

Calculating perplexity:

$$P = 2^{-\frac{1}{3} \times (\log_2(0.5) + \log_2(0.6) + \log_2(0.4))} = 2^{-\frac{1}{3} \times ((-1) + (-0.737) + (-1.322))} \approx 2.028$$

Calculation for Response 2: “chatbot like I”

For this 3-word response, we need the following probabilities:

$$P(\text{chatbot}|\langle \text{start} \rangle) = 0.3$$

$$P(\text{like}|\text{chatbot}) = 0.2$$

$$P(I|\text{chatbot like}) = 0.1$$

Calculating perplexity:

$$P = 2^{-\frac{1}{3} \times (\log_2(0.3) + \log_2(0.2) + \log_2(0.1))} = 2^{-\frac{1}{3} \times ((-1.737) + (-2.322) + (-3.322))} \approx 5.503$$

Comparison and Conclusion

Response	Text	Perplexity
Response 1	“I like chatbot”	2.028
Response 2	“chatbot like I”	5.503

In perplexity evaluation, **lower values indicate better performance**. Therefore, Response 1 (“I like chatbot”) is better according to this metric, as it has a lower perplexity value. This means that the sequence “I like chatbot” is more probable or natural according to the language model’s probability distribution than “chatbot like I”.

4. Multimodal Chatbots and Personality Design

a)

(i) Multi-modal Inputs/Outputs in Customer Service Chatbots

Multi-modal interactions enhance chatbot effectiveness by enabling richer and more natural communication.

- **Text, Speech, and Image Inputs:** Users can type, speak, or upload images, making interactions more accessible and intuitive. For instance, a customer troubleshooting a product issue can describe the problem verbally and send a picture for better assistance.
- **Adaptive Outputs:** The chatbot can respond in different formats—text for quick replies, voice for accessibility, and images (e.g., annotated screen-shots) for clearer explanations.
- **Improved Accuracy:** Combining modalities allows the chatbot to cross-validate information, reducing errors in understanding user queries.

(ii) User State Modeling for Personalized Responses

User state modeling enables the chatbot to adapt based on sentiment, conversation history, and engagement level.

- **Sentiment Analysis:** If the user is frustrated, the chatbot can adopt a more empathetic tone.
- **Conversation History:** The chatbot remembers past interactions, avoiding repetitive questions and offering continuity in discussions.
- **Engagement Level:** If the user is disengaged (e.g., short responses), the chatbot can switch to a more interactive approach.

Example Scenario: If a user is frustrated about a delayed order, the chatbot can shift to an apologetic tone, offer compensation, or suggest a human agent if engagement drops.

b) Multi-modal fusion combines different input modalities (e.g., speech, facial expressions, physiological signals) to improve emotion recognition accuracy. The three main fusion strategies are:

1. Early Fusion

- **Description:** Combines raw features from different modalities before feeding them into a classifier.
- **Advantages:** Captures cross-modal correlations early, leading to more comprehensive feature representations.

- **Challenges:** Requires aligned and synchronized data, making preprocessing complex.

2. Late Fusion

- **Description:** Each modality is processed separately with its own classifier, and their outputs (e.g., classification probabilities) are combined to make the final decision.
- **Advantages:** More flexible, allows independent processing of different modalities.
- **Challenges:** May lose fine-grained cross-modal interactions, leading to suboptimal performance in some cases.

3. Hybrid Fusion

- **Description:** Integrates features at multiple levels, leveraging both early and late fusion benefits.
- **Advantages:** Balances cross-modal feature extraction and decision-making, improving robustness.
- **Challenges:** More computationally expensive and requires complex model architectures.

By selecting the appropriate fusion strategy, emotion recognition systems can enhance accuracy and robustness, leading to better user experience in applications such as affective computing and human-computer interaction.

c) The effectiveness of a mental health chatbot depends on its personality traits, which should be carefully designed to ensure emotional sensitivity and user trust. Using the five-factor model, the appropriate levels for each factor are:

1. Artificiality: Low to Moderate

- Users should feel they are interacting with a supportive entity, but not be deceived into thinking it is human.
- A balance between artificial and natural responses maintains trust and avoids uncanny interactions.

2. Conscientiousness: Very High

- The chatbot must be reliable, structured, and consistent in offering support and guidance.
- Ensures accurate delivery of mental health resources and crisis intervention when needed.

3. **Vibrancy: Moderate**

- A calm and steady tone is preferred over excessive enthusiasm, which might feel inappropriate for users in distress.
- Some vibrancy is useful for maintaining engagement in long-term interactions.

4. **Civility: Very High**

- The chatbot should be empathetic, non-judgmental, and supportive at all times.
- Ensures a safe space where users feel comfortable expressing their emotions.

5. **Neuroticism: Very Low**

- The chatbot should remain emotionally stable, avoiding reactions that might escalate a user's distress.
- A composed and reassuring presence is crucial for effective mental health support.

A chatbot designed with these traits fosters trust, emotional safety, and effective mental health support, ensuring users receive compassionate and reliable assistance.